FPGA Implementation of the RGB LED Controller

This lab implements in VHDL the pulse width modulation (PWM) circuit designed on paper in Exercise 2 and tests the resulting RGB LED controller on the PYNQ-Z2 board. Exercise 2 and Lab 1 are prerequisites for this lab.

Hand-in instructions: Prepare a small report with block diagrams. Submit the report as a PDF with the source code files through the lecture moodle per the moodle submission deadline.

Important information ⚠

Please check the FPGA, VHDL, virtual machine and Windows points in this box before you start a lab or when you have issues.

FPGA:

- Remember to use the reset button/switch on the FPGA to reset your design and to turn this off again if using a switch.

VHDL:

- For combinational logic, use `process(all)`. Do not write your own sensitivity lists. Please remember to change files using `process(all)` to VHDL 2008. This is done by selecting the file in the `Source` tab. Look at the `Source File Properties` tab and change the type to `VHDL 2008` by clicking on the 3 dots in the `Type` field.
- For defining registers, use the clocked process style `process(CLKxCI, RSTxRI)`. Do not define combinational logic like `CNTxDP <= CNTxDP + AxDI` inside a clocked process. See Task 4 in Exercise 3 and its solution for further explanation.
- Never write to the same signal in multiple concurrent statements. This means that if you assign to a signal in a process that signal can only be assigned to in that process and nowhere else. The only place you are allowed to assign multiple times to a signal is inside the (single) process where it is assigned to.

Virtual machines:

- EDA server users must start Vivado with `vivado -source load_board_files.tcl` as described in Lab 1. If you do not see the board files in the Vivado GUI, you have likely used the command with a spelling error or something similar.

Windows users:

- Avoid spaces and special characters in your filepaths. Vivado projects can become corrupted if you have spaces and special characters in your filepaths.
- You may have to disable your antivirus tool before running simulations in Vivado.

For common questions/hints to this lab, please see the last page of this document which contains various hints and best-practices.

VHDL Implementation of the RGB LED Controller

Download the handout `.zip` file from Moodle. The handout contains three VHDL files:

- `pwm.vhdl`: template for the PWM circuit (`pwm`), which generates the PWM pulse that drives one color of the RGB LED.
- `toplevel.vhdl`: template for the RGB LED controller (`toplevel`), which instantiates the PWM circuit three times, once per color of the RGB LED.
- `pwm_tb.vhdl`: the testbench of the PWM circuit, which measures the width of the PWM pulse after each press of the push button and reports whether the measured width is correct.

Task 1: VHDL Implementation of the PWM Circuit

Implement the PWM circuit of Exercise 2 in `pwm.vhdl`. The input `PushxSI` comes from a push button and the output `LedxSO` drives one color of the RGB LED. The implementation must follow the block diagrams drawn in Exercise 2. The PWM counter and the threshold counter are both 18 bits wide, and every press of the push button increments the threshold counter by $2^{15} = 32768$.

Circuit designer's toolbox

Exercises 1b and 2 covered the first two steps of problem solving: understanding the problem at hand and devising a plan. This lab carries out that plan in VHDL. Problem solving splits into four steps:

1. **Understanding the problem:** writing out the ports and the purpose of each port, drawing a timing diagram, and anything else that builds an understanding of the problem. A student who starts writing code before the problem is understood often reaches halfway and then has to start over.
2. **Devising a plan:** in circuit design, drawing a schematic before writing any code. The second step overlaps with the first step.
3. **Carrying out the plan:** writing the VHDL code that implements the circuit and verifying that the code behaves as the schematic describes.
4. **Looking back:** reviewing how the problem was solved, whether a simpler solution exists, and how others solved the same problem. This course covers the fourth step in class by going through submitted solutions.

The four steps may look trivial. Skipping the first two steps is a habit worth unlearning early.

Task 2: Simulation of the PWM Circuit

Add the provided testbench `pwm_tb.vhdl` to the project in the same way as the testbench of Lab 1. Only `pwm_tb.vhdl` may be listed as a top-level file for simulation.

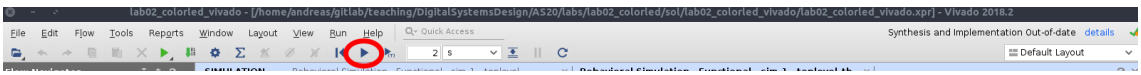


Figure 1: Top of the Vivado window during simulation, with the **Run All** button circled in red.

Pressing **Run All**, circled in red in Figure 1, runs the simulation to completion. The testbench applies seven presses of the push button and measures the width of the PWM pulse after each

press. For press n, with n running from 1 to 7, the testbench compares the measured width against the expected width of $n \times 32768$ clock cycles and prints the outcome to the Tcl Console, as shown in Figure 2. Inspect the waveforms in the waveform viewer as well, to confirm that the PWM counter, the threshold counter and the output pulse behave as the block diagrams of Exercise 2 describe.

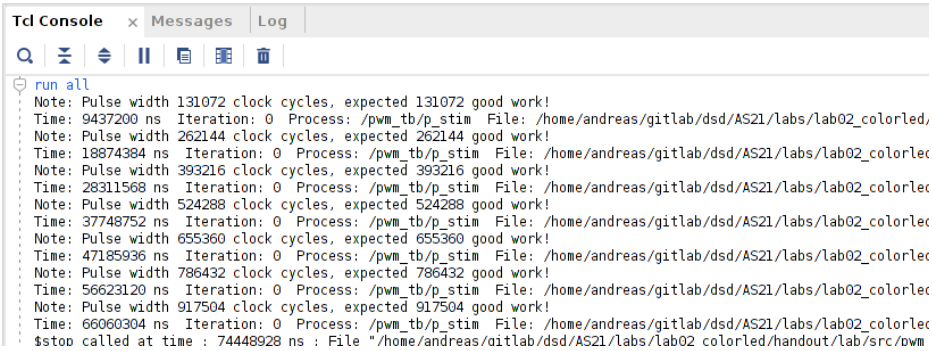


Figure 2: Tcl Console output of the testbench `pwm_tb.vhdl`. Each line reports the measured pulse width in clock cycles, the expected pulse width, and whether the two widths match.

Hints and common errors

By default the waveform viewer shows only the signals of the top-level entity of the simulation, which is not enough to locate a fault inside a sub-component of a larger design. The Vivado simulator can plot any signal of the design, as shown in Figure 3. Three steps are needed:

1. In the `Scope` tab, select the entity that holds the signals of interest.
2. Drag the signals of interest from the `Objects` tab into the waveform viewer.
3. Restart the simulation. A signal added after the simulation has run carries no recorded history, so the waveform appears only after a restart.

Task 3: Instantiation of the PWM Circuit in the RGB LED Controller

Implement the RGB LED controller in `toplevel.vhdl` by instantiating the PWM circuit three times, once per color. The component declaration of `pwm` is already present in the template, so only the port maps that tie each instance to the matching ports of `toplevel` remain to be written.

Implementation of the RGB LED Controller on the PYNQ-Z2 Board

Task 4: Constraint File of the PYNQ-Z2 Board

Launch Vivado and create a new project named `work_lab02_colorled`, which is the work directory supplied in the handout. Add `pwm.vhdl`, `toplevel.vhdl` and `pwm_tb.vhdl` to the project. Then add the constraint file `PYNQ-Z2_v1.0.xdc`, which sits in the `constr` directory of the handout as in Lab 1 and is also available on Moodle. The provided `.xdc` file is the standard template of the PYNQ-Z2 board, so every line of the file is commented out. Uncomment and edit four groups of constraints, so that the port names match the ports of `toplevel`:

- Connect `CLKxCI` to pin Y7, which carries the $25\,\mathrm{MHz}$ CMOS oscillator of the class interface PCB through the Raspberry Pi header.
- Connect `RSTxRI` to BTN0.

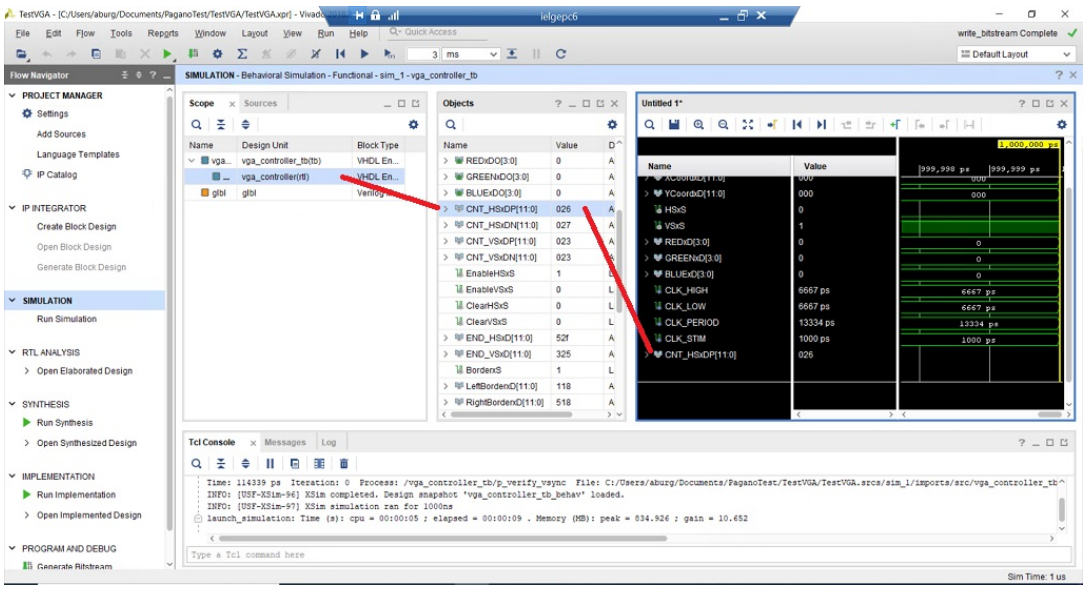


Figure 3: Adding an internal signal to the waveform viewer of the Vivado simulator. The `Scope` tab unfolds the top-level entity into the sub-components of the design, and any signal listed in the `Objects` tab can be dragged into the waveform viewer, as marked by the red arrow. A signal added after the simulation has run carries no recorded history, so the simulation has to be restarted before the waveform appears.

- Connect `PushRedxSI` to BTN3, `PushGreenxSI` to BTN2 and `PushBluexSI` to BTN1.
- Connect `LedRedxSO`, `LedGreenxSO` and `LedBluexSO` to LED4, which is an RGB LED with one input per color.

The `.xdc` file of Lab 1 shows what an edited pin constraint looks like.

Task 5: Programming the PYNQ-Z2 Board

Run synthesis and implementation, generate the bitstream and program the FPGA. Three checks verify the RGB LED controller on the PYNQ-Z2 board:

- Press BTN0 to reset every counter. The duty cycle of every color then drops to 0 and LED4 stays off (black).
- Press BTN3 to raise the intensity of red. Press BTN0 again, then repeat the check for green on BTN2 and for blue on BTN1.
- Mix the three colors according to the RGB color code of a target color and check that LED4 shows that color.

Common Questions

Common questions/remarks for this lab are:

- **How do I create a VHDL file?** After creating a project in Vivado you can click `File` → `Add Sources` → `Add or create design sources`. Alternatively, just use your normal code editor and create new files with the `.vhdl` (recommended) or `.vhd` extensions.
- **How do I resolve the warning 'The PS7 cell must be used in this Zynq design ...?** This warning can be safely ignored as it's unrelated to what we do on the FPGA.

- **How do I resolve the error 'Unconstrained Logical Port'?** While VHDL itself is case-insensitive, `.xdc` **constraints are case sensitive** and your port names should match those in the `.xdc` file in case as well.
- **Outlook does not allow `.vhd` files:** You cannot send `.vhd` files in Outlook, try renaming to `.vhdl` as the `.vhd` extension is also used for **V**irtual **H**ard **D**isk on Windows.
- **Remember that order matters in processes!** Since the order of assignments are done sequentially in a process, meaning that in the example below `DxSO` is only assigned `AxSI and BxSI` and never `AxSI or BxSI`.

Listing 1: This implementation ignores the line `AxSI or BxSI` as it is always overwritten by the final assignment to `DxSO`.

```
process(all)
begin
  if (CxSO = '0') then
    DxSO <= AxSI or BxSI;
  end if;

  CxSO <= not AxSI;
  DxSO <= AxSI and BxSI;
end process;
```

- **Remember to separate the description of the flip-flops from the combinational logic!** Use a single process for updating the flip-flops and a separate process or concurrent assignments for updating the adder as shown below. This is really important! We also discuss this in Exercise 3.

Listing 2: VHDL code to show how to define flip-flops for a counter.

```
CNTxDN <= CNTxDP + 1; -- Increment outside clock-process

process(CLKxCI, RSTxRI)
begin
  if (RSTxRI = '1') then
    CNTxDP <= (others => '0');
  elsif CLKxCI'event and CLKxCI = '1' then
    CNTxDP <= CNTxDN;
  end if;
end process;
```

- **Remember to never write to the same signal in multiple concurrent statements!** When assigning to a signal in a process, you can only assign to that signal in that (single) process. The code below in Listing 3 shows the signal `CNTxDN` being assigned to in two different concurrent statements, which is not allowed. With this code, you will see an 'X' for `CNTxDN` in the waveform viewer. The solution is to put the default assignment in a process like shown in Listing 4.

Listing 3: VHDL code which shows how not to assign to a signal!

```
CNTxDN <= CNTxDP;

process(all)
begin
  if (In0xSI = '1') then
    CNTxDN <= CNTxDP + 1;
  end if;
end process;
```

- **Use default values in your process!** To avoid introducing errors in your code from missing assignments to signals, you should always use a default value for all signals assigned to in a `process(all)` when describing combinational logic as shown in Listing 4. This is done as the first thing in a process.

Listing 4: VHDL code which shows the assignment of a default value.

```vhdl
process(all)
begin
  -- Default values
  CNTxDN <= CNTxDP;
  AxD    <= (others => '0');
  BxD    <= (others => '0');

  -- Actual logic after default values
  if (In0xSI = '1') then
    CNTxDN <= CNTxDP + 1;
    AxD    <= In1xSI;
  elsif (In1xSI = '1') then
    CNTxDN <= CNTxDP - 1;
    BxD    <= In1xSI;
  end if;
end process;
```